ИНСТИТУТ ТЕХНОЛОГИЙ УПРАВЛЕНИЯ
Центр технологий 1С

Дисциплина: «Проектирование и архитектура программного обеспечения»

ПРАКТИЧЕСКОЕ ЗАДАНИЕ

«Написание документации проекта. UML-диаграммы»

Тема проекта № 12:

«Генератор отчётов на основе SQL-подобного языка»

(рабочее название — SQLReport)

Спецификация требований к программному обеспечению (SRS)

Выполнили:	студенты группы ЭПБО-02-25 Батырбеков Ыйман Адилетович Рабаданов Абудулманап Султанмутович
Преподаватель:	Яковлев Михаил Сергеевич
Семестр:	1 семестр, 2026–2027 уч. г.

Москва, 2026

СОДЕРЖАНИЕ

1. Введение.....3
1.1. Цель.....3
1.2. Область действия (Scope).....3
1.3. Определения и аббревиатуры.....4
1.4. Ссылки.....5
2. Общее описание.....5
2.1. Описание продукта.....5
2.2. Функции продукта.....6
2.3. Характеристики пользователей.....6
2.4. Ограничения.....7
2.5. Допущения и зависимости.....7
3. Детальные (специфические) требования.....7
3.1. Методология сбора требований.....7
3.2. Бизнес-требования.....9
3.3. Пользовательские требования.....10
3.4. Функциональные требования.....11
3.5. Нефункциональные требования.....13
3.6. Грамматика языка запросов.....14
3.7. Внешние интерфейсы.....15
4. Диаграммы UML.....17
4.1. Диаграмма вариантов использования.....17
4.2. Диаграмма классов.....19
4.3. Диаграмма последовательности.....22

1. Введение

1.1. Цель

Настоящий документ содержит спецификацию требований к программному обеспечению «Генератор отчётов на основе SQL-подобного языка» (SQLReport). Он определяет назначение, функции, ограничения и атрибуты качества продукта и служит техническим заданием для команды разработки, основой для тестирования и приёмки. Структура документа опирается на рекомендации IEEE 830 / ISO/IEC/IEEE 29148, а требования классифицированы по модели К. Вигерса.

Целевая аудитория документа: заказчик (преподаватель), разработчики, тестировщики, а также разработчики-интеграторы, которые будут использовать программный интерфейс библиотеки.

1.2. Область действия (Scope)

SQLReport — библиотека и консольное приложение на Java для формирования табличных отчётов по CSV-файлам с помощью SQL-подобного языка запросов. Продукт позволяет выполнять выборку, фильтрацию, соединение, группировку с агрегатными функциями, сортировку и ограничение результата и выводить отчёт в консоль или в файлы форматов CSV, HTML и Markdown.

В область действия входит:

- чтение CSV-файлов; оператор SELECT с WHERE, JOIN, GROUP BY, HAVING, ORDER BY, LIMIT;
- агрегатные функции COUNT, SUM, AVG, MIN, MAX и механизм добавления новых;
- консольный интерфейс (REPL, однократный запрос, скрипт) и Java API;
- вывод в консольную таблицу, CSV, HTML, Markdown.

Вне области действия:

- изменение данных (INSERT, UPDATE, DELETE), создание таблиц, транзакции;
- подзапросы, оконные функции, UNION;
- графический интерфейс и построение графиков;
- сетевой режим (сервер, многопользовательский доступ).

Польза для заказчика: сокращение времени подготовки типовых отчётов, их воспроизводимость, отсутствие необходимости в СУБД и возможность встраивания в другие Java-приложения (подробно — бизнес-требования BR-1…BR-5, раздел 3.2).

1.3. Определения и аббревиатуры

Таблица 1 — Определения и аббревиатуры

Термин	Определение
CSV	Comma-Separated Values — текстовый формат табличных данных, где значения разделены запятыми (или другим разделителем), RFC 4180.
SQL	Structured Query Language — язык запросов к реляционным данным; в проекте используется его подмножество (оператор SELECT).
SRS	Software Requirements Specification — спецификация требований к программному обеспечению.
CLI	Command Line Interface — интерфейс командной строки.
REPL	Read–Eval–Print Loop — интерактивный режим «прочитать запрос — выполнить — вывести результат».
API	Application Programming Interface — программный интерфейс библиотеки для вызова из другого кода.
JRE / JVM	Java Runtime Environment / Java Virtual Machine — среда выполнения Java-программ.
Лексер	Компонент, разбивающий текст запроса на лексемы (токены): ключевые слова, идентификаторы, числа, операторы.
Парсер	Компонент, строящий по последовательности токенов абстрактное синтаксическое дерево методом рекурсивного спуска.
AST	Abstract Syntax Tree — абстрактное синтаксическое дерево запроса.
Оператор (плана)	Узел плана выполнения (Scan, Filter, Join, Aggregate, Sort, Project, Limit), выдающий строки по одной через метод next().
Агрегатная функция	Функция, вычисляющая одно значение по множеству строк группы: COUNT, SUM, AVG, MIN, MAX.
Отчёт	Результат выполнения запроса, представленный в одном из форматов вывода.
NULL	Отсутствующее значение; в CSV соответствует пустому полю.

Термин	Определение
BR / US / FR / NFR	Идентификаторы бизнес-требований, пользовательских историй, функциональных и нефункциональных требований.

1.4. Ссылки

- ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering;
- IEEE Std 830-1998. Recommended Practice for Software Requirements Specifications;
- Вигерс К., Битти Дж. Разработка требований к программному обеспечению. 3-е изд.;
- RFC 4180. Common Format and MIME Type for Comma-Separated Values (CSV) Files;
- ISO/IEC 9075 (SQL) — в части оператора SELECT;
- OMG Unified Modeling Language (UML), версия 2.5.1.

2. Общее описание

2.1. Описание продукта

SQLReport — новый самостоятельный продукт. Он работает локально на компьютере пользователя, взаимодействует с файловой системой (чтение CSV, запись отчётов) и не требует СУБД или сетевых сервисов. Продукт поставляется в виде одного JAR-файла, который можно запустить как консольное приложение или подключить к другому Java-проекту как библиотеку.

Обработка запроса организована как конвейер из независимых слоёв:

```
текст запроса → Lexer → токены → Parser → AST (SelectStatement)
      → QueryPlanner (+ TableCatalog / CsvReader) → дерево операторов
      → Scan → Join → Filter → Aggregate → Sort → Project → Limit
      → ResultTable → ReportFormatter → отчёт (консоль / CSV / HTML /
Markdown)
```

Каждый слой зависит только от нижележащих, что позволяет тестировать их по отдельности и заменять реализации (например, добавить

новый формат вывода, не затрагивая парсер). Подробная структура классов приведена в разделе 4.2.

2.2. Функции продукта

Основные функции продукта (без детализации):

- загрузка CSV-файлов с автоопределением типов столбцов, разделителя и поддержкой кодировок;
- разбор и проверка запросов на SQL-подобном языке с диагностикой ошибок;
- выполнение запросов: WHERE, JOIN, GROUP BY с агрегатными функциями, HAVING, ORDER BY, LIMIT, DISTINCT;
- просмотр структуры CSV-файла (DESCRIBE) и встроенная справка (HELP);
- вывод отчёта в виде консольной таблицы и экспорт в CSV, HTML, Markdown с сохранением в файл;
- три режима работы CLI: интерактивный, однократный запрос, выполнение скрипта;
- программный интерфейс для Java-приложений и регистрация пользовательских агрегатных функций.

2.3. Характеристики пользователей

Таблица 2 — Классы пользователей

Класс пользователей	**Характеристика**	**Частота использования**	**Интерфейс**
Аналитик данных	Знает базовый SQL (SELECT, WHERE, GROUP BY), уверенно работает с Excel, умеет запускать программы из командной строки. Не программирует на Java.	Часто (ежедневно/еженедельно)	CLI: REPL, -q, -f
Разработчик-интегратор	Java-разработчик среднего уровня, подключает библиотеку как зависимость, читает Javadoc.	При интеграции, далее — автоматически	Java API
Преподаватель / проверяющий	Эксперт; оценивает архитектуру, документацию, тесты, запускает примеры.	Эпизодически	CLI, исходный код

2.4. Ограничения

- язык реализации — Java 17; использование библиотек допустимо, но они не должны подменять реализацию проекта (см. NFR-14);
- данные — только локальные CSV-файлы, доступные на чтение;
- весь набор строк обрабатываемых таблиц может загружаться в оперативную память (потоковая обработка — для операторов Scan, Filter, Project, Limit);
- срок разработки — один семестр, команда из 1–3 человек.

2.5. Допущения и зависимости

- CSV-файлы имеют одинаковое число полей в каждой строке; строки с иным числом полей считаются ошибкой формата с указанием номера строки;
- пользователь знаком с базовым SQL; обучение языку запросов не входит в задачи продукта;
- результаты сверяются с SQLite 3 как эталоном поведения для поддерживаемого подмножества.

3. Детальные (специфические) требования

3.1. Методология сбора требований

Для сбора требований использована комбинация традиционных и agile-ориентированных методов. Такой выбор обусловлен тем, что у продукта есть готовые аналоги (их анализ даёт «нижнюю планку» ожиданий), есть конкретные представители пользователей (их потребности выясняются в интервью), а разработка ведётся итеративно (story map позволяет разбить объём на релизы).

Таблица 3 — Использованные методы сбора требований

Метод	**Что использовалось**	**Результат**
Анализ документов и существующих решений	RFC 4180, подмножество стандарта SQL, документация SQLite, csvkit, q, DuckDB; текст задания курса	Ограничения и обязательные возможности, понятийный аппарат, перечень ожидаемых конструкций языка
Интервью	Полуструктурированное интервью с представителем заказчика и потенциальными пользователями (аналитик, разработчик) в рамках учебного сценария	Реальные сценарии использования, объёмы данных, требования к ошибкам и форматам вывода
User Stories	Формулирование потребностей от лица ролей с критериями приёмки	12 пользовательских историй US-01…US-12
Story Mapping	Раскладка историй по шагам пользовательского пути и по релизам	План релизов: MVP, релиз 2, релиз 3

Подробные материалы (сравнение аналогов, протокол интервью, карта историй) приведены в отчёте по практическому заданию «Выбор темы проекта. Сбор требований»; ниже дана их сводка.

Анализ аналогов показал, что существующие решения либо требуют ручной работы (табличные редакторы), либо внешнего окружения (Python, СУБД) и не встраиваются в Java-приложение как библиотека. **Интервью** выявило основные сценарии: сводки с GROUP BY и агрегатами, топ-N, соединение с справочником, вывод в консоль/CSV/HTML/Markdown, понятные сообщения об ошибках. **Story Map** определил порядок реализации:

Таблица 4 — Карта пользовательских историй (Story Map)

Релиз	**Подготовить данные**	**Написать запрос**	**Выполнить запрос**	**Получить отчёт**	**Автоматизировать**
MVP (релиз 1)	Указать CSV-файл с заголовком; автоопределение типов	SELECT, FROM, WHERE, ORDER BY, LIMIT; псевдонимы AS	Выполнить запрос в REPL, получить ошибку с позицией	Таблица в консоли	Запуск с параметром -q "запрос"
Релиз 2	Разделитель, кодировка, файл без заголовка	GROUP BY, HAVING, COUNT/SUM/AVG/MIN/MAX, DISTINCT	DESCRIBE файла, справка HELP	Вывод в CSV / Markdown, сохранение в файл	Скрипт из файла (-f), Java API

Релиз	Подготовить данные	Написать запрос	Выполнить запрос	Получить отчёт	Автоматизировать
Релиз 3	Кеширование загруженных таблиц	INNER/LEFT JOIN, выражения и функции в SELECT	Статистика: время выполнения, число строк	Вывод в HTML	Регистрация собственных агрегатных функций

3.2. Бизнес-требования

Бизнес-требования описывают, **зачем** создаётся продукт и какую пользу он должен принести заказчику и пользователям. Они являются верхним уровнем иерархии Вигерса: каждое требование нижних уровней должно поддерживать хотя бы одно бизнес-требование.

Таблица 5 — Бизнес-требования

ID	Бизнес-требование
BR-1	Сократить время подготовки типового сводного отчёта по CSV-выгрузке с десятков минут ручной работы в табличном редакторе до одной команды (не более 1 минуты, включая написание запроса знакомым с SQL пользователем).
BR-2	Обеспечить воспроизводимость отчётов: один и тот же текст запроса на тех же данных всегда даёт один и тот же результат и может быть сохранён в файл-скрипт для повторного запуска.
BR-3	Исключить необходимость установки и администрирования СУБД: для работы достаточно среды выполнения Java (JRE 17+) и исполняемого JAR-файла.
BR-4	Дать разработчикам возможность встраивать генерацию отчётов в собственные Java-приложения через программный интерфейс (библиотеку).
BR-5	Учебная цель: продемонстрировать полный цикл разработки — требования, SRS, UML, многоуровневую архитектуру интерпретатора запросов, тестирование — в рамках одного семестра.

Критерии успеха проекта:

- типовые запросы из раздела «Примеры» выполняются и дают результат, совпадающий с SQLite на тех же данных;
- отчёт по файлу из 1 млн строк строится не дольше 5 секунд (см. NFR-1);
- пользователь, знающий базовый SQL, выполняет первый запрос без чтения документации, пользуясь только командой HELP.

Границы проекта (входит в объём):

- чтение CSV-файлов; оператор SELECT с WHERE, JOIN, GROUP BY, HAVING, ORDER BY, LIMIT;

- агрегатные функции COUNT, SUM, AVG, MIN, MAX и механизм добавления новых;
- консольный интерфейс (REPL, однократный запрос, скрипт) и Java API;
- вывод в консольную таблицу, CSV, HTML, Markdown.

Не входит в объём (явные исключения):

- изменение данных (INSERT, UPDATE, DELETE), создание таблиц, транзакции;
- подзапросы, оконные функции, UNION;
- графический интерфейс и построение графиков;
- сетевой режим (сервер, многопользовательский доступ).

3.3. Пользовательские требования

Пользовательские требования описывают, **что** пользователи должны иметь возможность делать с помощью продукта. Они записаны в формате пользовательских историй «Как <роль>, я хочу <действие>, чтобы <ценность>» с критериями приёмки.

Таблица 6 — Пользовательские требования (User Stories)

ID	Пользовательская история	Критерий приёмки
US-01	Как аналитик, я хочу выполнить SELECT-запрос к CSV-файлу, указав имя файла в FROM, чтобы получить нужные столбцы без открытия Excel.	Запрос SELECT name, grade FROM students.csv выводит два столбца и все строки файла.
US-02	Как аналитик, я хочу отфильтровать строки условием WHERE (сравнения, AND/OR/NOT, LIKE, IN, BETWEEN, IS NULL), чтобы работать только с нужным подмножеством данных.	WHERE grade >= 4 AND group = 'ИТ-21' оставляет только подходящие строки.
US-03	Как аналитик, я хочу сгруппировать строки (GROUP BY) и посчитать агрегаты COUNT, SUM, AVG, MIN, MAX, чтобы получать сводные показатели.	Результат совпадает с аналогичным запросом в SQLite на тех же данных.
US-04	Как аналитик, я хочу отфильтровать группы (HAVING), отсортировать (ORDER BY) и ограничить вывод (LIMIT), чтобы строить рейтинги и топ-N.	ORDER BY avg_grade DESC LIMIT 3 выводит три строки с наибольшим средним.
US-05	Как аналитик, я хочу соединить два CSV-файла (JOIN … ON), чтобы дополнять данные справочниками.	INNER и LEFT JOIN дают те же строки, что и SQLite.

ID	Пользовательская история	Критерий приёмки
US-06	Как аналитик, я хочу получить понятное сообщение об ошибке с номером строки и позиции, чтобы быстро исправить опечатку в запросе.	Для SELEC name … выводится «Синтаксическая ошибка (1:1): ожидалось SELECT»; программа продолжает работу.
US-07	Как аналитик, я хочу просмотреть структуру файла (DESCRIBE students.csv), чтобы узнать имена и типы столбцов перед написанием запроса.	Выводится список столбцов с выведенными типами и число строк.
US-08	Как аналитик, я хочу выбрать формат вывода (таблица, CSV, HTML, Markdown) и сохранить отчёт в файл, чтобы использовать результат в письме, вики или другой программе.	Опции --format html --out report.html создают корректный HTML-файл.
US-09	Как аналитик, я хочу указать разделитель, кодировку и наличие заголовка в CSV, чтобы работать с выгрузками из разных систем.	Файл в Windows-1251 с разделителем «;» читается без искажений.
US-10	Как аналитик, я хочу выполнить скрипт из нескольких запросов из файла, чтобы повторять регулярные отчёты одной командой.	Команда -f weekly.sql выполняет все запросы по порядку и выводит результаты.
US-11	Как разработчик-интегратор, я хочу вызвать движок из Java-кода (engine.execute(sql)) и получить объект с результатом, чтобы встраивать отчёты в своё приложение.	Метод возвращает ResultTable со схемой и строками; ошибки — проверяемыми исключениями QueryException.
US-12	Как разработчик-интегратор, я хочу зарегистрировать собственную агрегатную функцию (например, MEDIAN), чтобы расширять язык без изменения исходного кода движка.	После registerFunction("median", Median::new) запрос с median(grade) выполняется.

3.4. Функциональные требования

Функциональные требования определяют поведение системы, которое необходимо реализовать, чтобы пользователи могли выполнить свои задачи. Каждое требование сформулировано проверяемо и связано с пользовательскими историями (столбец «Источник»).

Таблица 7 — Функциональные требования

ID	Функциональное требование	Источник
Загрузка данных (CSV)		
FR-01	Система должна читать CSV-файлы в соответствии с RFC 4180 (поля в кавычках, экранированные кавычки, переводы строк внутри полей).	US-01

ID	Функциональное требование	Источник
FR-02	Система должна поддерживать задание разделителя (по умолчанию «,»; автоопределение «,» / «;» / табуляции по первой строке), кодировки (UTF-8 по умолчанию, Windows-1251) и признака наличия строки заголовка.	US-09
FR-03	Система должна автоматически определять тип столбца (INTEGER, DECIMAL, BOOLEAN, DATE в формате ISO-8601, STRING) по значениям столбца; пустое поле интерпретируется как NULL.	US-01, US-07
FR-04	Имя таблицы в FROM/JOIN задаётся путём к файлу относительно рабочего каталога; допускается псевдоним (FROM students.csv AS s).	US-01, US-05
FR-05	Повторно используемые файлы должны кешироваться в пределах сеанса; кеш сбрасывается, если файл изменился (по дате модификации).	US-10
Язык запросов		
FR-06	Система должна разбирать оператор SELECT вида: SELECT [DISTINCT] список FROM таблица [JOIN …] [WHERE …] [GROUP BY …] [HAVING …] [ORDER BY … [ASC|DESC]] [LIMIT n [OFFSET m]]. Грамматика приведена в разделе 3.6.	US-01…US-05
FR-07	Ключевые слова и имена функций нечувствительны к регистру; имена столбцов с пробелами задаются в двойных кавычках, строковые литералы — в одинарных.	US-01
FR-08	В выражениях поддерживаются: арифметика (+, −, *, /, %), сравнения (=, <>, <, <=, >, >=), логические AND, OR, NOT, а также LIKE, IN, BETWEEN, IS [NOT] NULL; скобки; трёхзначная логика при NULL.	US-02
FR-09	Поддерживаются агрегатные функции COUNT(*), COUNT(expr), COUNT(DISTINCT expr), SUM, AVG, MIN, MAX; агрегаты по пустой группе возвращают NULL (кроме COUNT = 0).	US-03
FR-10	Поддерживаются INNER JOIN и LEFT JOIN с условием ON (минимум — равенство столбцов).	US-05
FR-11	Поддерживаются служебные команды: DESCRIBE <файл>, HELP [тема], EXIT.	US-07
Выполнение запросов		
FR-12	Перед выполнением система должна проверять семантику: существование файлов и столбцов, однозначность имён при JOIN, допустимость столбцов в SELECT при GROUP BY, совместимость типов в выражениях.	US-06
FR-13	Логический порядок выполнения: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET.	US-03, US-04
FR-14	Сортировка должна быть стабильной; NULL при сортировке по возрастанию располагаются в конце.	US-04
FR-15	Результат запроса представляется объектом ResultTable (схема + строки), не зависящим от формата вывода.	US-11
Формирование отчёта		

ID	Функциональное требование	Источник
FR-16	Система должна выводить результат в виде выровненной текстовой таблицы с заголовком и итоговой строкой «N строк, t мс»; дробные значения по умолчанию округляются до 2 знаков после запятой (настраивается опцией --precision).	US-01
FR-17	Система должна экспортировать результат в форматы CSV (RFC 4180), HTML (таблица с экранированием спецсимволов) и Markdown.	US-08
FR-18	Отчёт может быть сохранён в файл (опция --out); существующий файл перезаписывается только с флагом --force.	US-08
Интерфейсы и расширяемость		
FR-19	Консольное приложение поддерживает три режима: интерактивный REPL (запрос завершается «;»), однократный запрос (-q) и выполнение скрипта (-f).	US-10
FR-20	Публичный Java API: класс ReportEngine с методами execute(String), describe(String), registerFunction(String, Supplier<AggregateFunction>).	US-11, US-12
FR-21	При ошибке система выводит тип ошибки, позицию (строка:столбец), фрагмент запроса с указателем «^» и не завершает сеанс REPL.	US-06

3.5. Нефункциональные требования

Нефункциональные требования задают атрибуты качества (производительность, надёжность, удобство, переносимость, сопровождаемость) и ограничения реализации. Для каждого атрибута указана измеримая характеристика, по которой требование можно проверить.

Таблица 8 — Нефункциональные требования

ID	Нефункциональное требование
Производительность	
NFR-1	Запрос с WHERE и GROUP BY по CSV-файлу объёмом 1 000 000 строк × 10 столбцов (~100 МБ) выполняется не более чем за 5 с на ПК с 4-ядерным процессором и SSD.
NFR-2	Время разбора запроса длиной до 2 000 символов — не более 50 мс.
NFR-3	Потребление памяти при обработке файла 100 МБ — не более 1 ГБ кучи JVM (-Xmx1g).
Надёжность	
NFR-4	Никакой входной запрос или некорректный CSV-файл не должен приводить к аварийному завершению с необработанным исключением; все ошибки преобразуются в QueryException с понятным сообщением.
NFR-5	Исходные CSV-файлы открываются только на чтение и никогда не изменяются.
Удобство использования	

ID	Нефункциональное требование
NFR-6	Сообщения интерфейса и об ошибках — на русском языке; каждое сообщение об ошибке содержит позицию и, где возможно, подсказку («возможно, вы имели в виду: grade»).
NFR-7	Команда HELP содержит краткую справку по синтаксису и не менее 5 примеров запросов.
Переносимость	
NFR-8	Работа на Windows 10+, Linux, macOS при наличии JRE 17 или новее; поставка — один исполняемый JAR-файл.
NFR-9	Корректная работа с путями и переводами строк всех трёх ОС (\n, \r\n).
Сопровождаемость и расширяемость	
NFR-10	Архитектура разделена на независимые пакеты parser, ast, data, execution, output; зависимости только «сверху вниз», без циклов.
NFR-11	Покрытие модульными тестами (JUnit 5) пакетов parser и execution — не менее 80 % строк.
NFR-12	Добавление новой агрегатной функции или формата вывода требует создания одного класса, реализующего интерфейс, без изменения существующего кода (принцип открытости/закрытости).
Ограничения реализации	
NFR-13	Язык реализации — Java 17; сборка — Gradle или Maven.
NFR-14	Запрещено использовать готовые SQL-движки и парсеры (H2, SQLite JDBC, Apache Calcite, JSqlParser, ANTLR-грамматики SQL) и готовые CSV-парсеры: лексер, парсер, исполнитель и чтение CSV реализуются самостоятельно. Допустимы JUnit 5 (тесты) и picocli (разбор аргументов командной строки).

3.6. Грамматика языка запросов

Синтаксис поддерживаемого подмножества SQL задан в расширенной форме Бэкуса — Наура (EBNF). Ключевые слова нечувствительны к регистру; IDENT — идентификатор, QUOTED_IDENT — идентификатор в двойных кавычках, FILE_NAME — имя или путь к CSV-файлу, STRING — строка в одинарных кавычках.

```
query        = select_stmt [ ";" ] ;
select_stmt  = "SELECT" [ "DISTINCT" ] select_list
               "FROM" table_ref { join_clause }
               [ "WHERE" expr ]
               [ "GROUP" "BY" expr { "," expr } ]
               [ "HAVING" expr ]
               [ "ORDER" "BY" order_item { "," order_item } ]
               [ "LIMIT" NUMBER [ "OFFSET" NUMBER ] ] ;
select_list  = "*" | select_item { "," select_item } ;
select_item  = expr [ [ "AS" ] IDENT ] ;
table_ref    = FILE_NAME [ [ "AS" ] IDENT ] ;
join_clause  = [ "INNER" | "LEFT" [ "OUTER" ] ] "JOIN" table_ref "ON" expr ;
order_item   = expr [ "ASC" | "DESC" ] ;
expr         = or_expr ;
```

```
or_expr      = and_expr { "OR" and_expr } ;
and_expr     = not_expr { "AND" not_expr } ;
not_expr     = [ "NOT" ] predicate ;
predicate    = additive [ comp_op additive
                        | [ "NOT" ] "LIKE" additive
                        | [ "NOT" ] "IN" "(" expr { "," expr } ")"
                        | [ "NOT" ] "BETWEEN" additive "AND" additive
                        | "IS" [ "NOT" ] "NULL" ] ;
comp_op      = "=" | "<>" | "!=" | "<" | "<=" | ">" | ">=" ;
additive     = term { ( "+" | "-" ) term } ;
term         = unary { ( "*" | "/" | "%" ) unary } ;
unary        = [ "-" ] primary ;
primary      = NUMBER | STRING | "NULL" | "TRUE" | "FALSE"
             | function_call | column_ref | "(" expr ")" ;
function_call= IDENT "(" ( "*" | [ "DISTINCT" ] expr { "," expr } ) ")" ;
column_ref   = [ IDENT "." ] ( IDENT | QUOTED_IDENT ) ;
```

Приоритет операций (от низшего к высшему): OR → AND → NOT → сравнения и предикаты → +, − → *, /, % → унарный минус. Грамматика однозначна и разбирается методом рекурсивного спуска с одним токеном предпросмотра (LL(1)).

3.7. Внешние интерфейсы

Интерфейс командной строки.

```
java -jar sqlreport.jar                         # интерактивный режим (REPL)
java -jar sqlreport.jar -q "SELECT ..." [опции]   # однократный запрос
java -jar sqlreport.jar -f weekly.sql  [опции]    # скрипт из файла

Опции:
  --format table|csv|html|md   формат вывода (по умолчанию table)
  --out <файл>                 сохранить отчёт в файл
  --force                      перезаписать существующий файл
  --delimiter <символ>         разделитель CSV (по умолчанию —
автоопределение)
  --encoding <кодировка>       кодировка CSV (по умолчанию UTF-8)
  --no-header                  в CSV нет строки заголовка (столбцы c1,
c2, ...)
  --precision <n>              число знаков после запятой (по умолчанию 2)
  --dir <каталог>              рабочий каталог для имён файлов в FROM
```

Программный интерфейс (Java API). Пример встраивания движка в приложение:

```
ReportEngine engine = ReportEngine.builder()
        .baseDir(Path.of("data"))
        .csvOptions(CsvOptions.defaults().delimiter(';'))
        .build();

engine.registerFunction("median", Median::new);      // своя агрегатная
функция

try {
    ResultTable result = engine.execute(
        "SELECT \"group\", median(grade) AS med FROM students.csv GROUP
BY \"group\"");
    String html = new HtmlFormatter().format(result);
    Files.writeString(Path.of("report.html"), html);
} catch (QueryException e) {
    System.err.println(e.getMessage());               // "Столбец 'grde' не
найден (1:24) ..."
}
```

Пример работы. Исходный файл students.csv:

```
name,group,subject,grade
Иванов,ИТ-21,Математика,5
Иванов,ИТ-21,Физика,4
Иванов,ИТ-21,Программирование,5
Петрова,ИТ-21,Математика,4
Петрова,ИТ-21,Физика,3
Петрова,ИТ-21,Программирование,5
Сидоров,ИТ-22,Математика,3
Сидоров,ИТ-22,Физика,2
Сидоров,ИТ-22,Программирование,4
Козлова,ИТ-22,Математика,5
Козлова,ИТ-22,Физика,5
Козлова,ИТ-22,Программирование,4
```

Запрос — средний балл студентов по оценкам не ниже 3, топ-3:

```
SELECT name, avg(grade) AS avg_grade, count(*) AS n
FROM students.csv
WHERE grade >= 3
GROUP BY name
ORDER BY avg_grade DESC
LIMIT 3;
```

Ожидаемый вывод (формат table; при равенстве avg_grade сохраняется порядок групп — FR-14):

```
+---------+-----------+---+
| name    | avg_grade | n |
+---------+-----------+---+
| Иванов  |      4.67 | 3 |
| Козлова |      4.67 | 3 |
| Петрова |      4.00 | 3 |
+---------+-----------+---+
3 строки, 4 мс
```

Пример сообщения об ошибке (FR-21):

```
sqlreport> SELECT name, avg(grde) FROM students.csv GROUP BY name;
Семантическая ошибка (1:21): столбец 'grde' не найден в students.csv
  SELECT name, avg(grde) FROM students.csv GROUP BY name;
                   ^^^^
  Возможно, вы имели в виду: grade
```

4. Диаграммы UML

4.1. Диаграмма вариантов использования

Диаграмма вариантов использования (рисунок 1) отражает функциональные требования системы. Основные акторы — Аналитик (работает через CLI) и Разработчик-интегратор (работает через Java API); вторичный актор — Файловая система, из которой читаются CSV-файлы и в которую сохраняются отчёты. Отношение «include» показывает обязательные под-сценарии (любой запрос разбирается и загружает данные), «extend» — условные расширения (сообщение об ошибке, сохранение отчёта в файл).

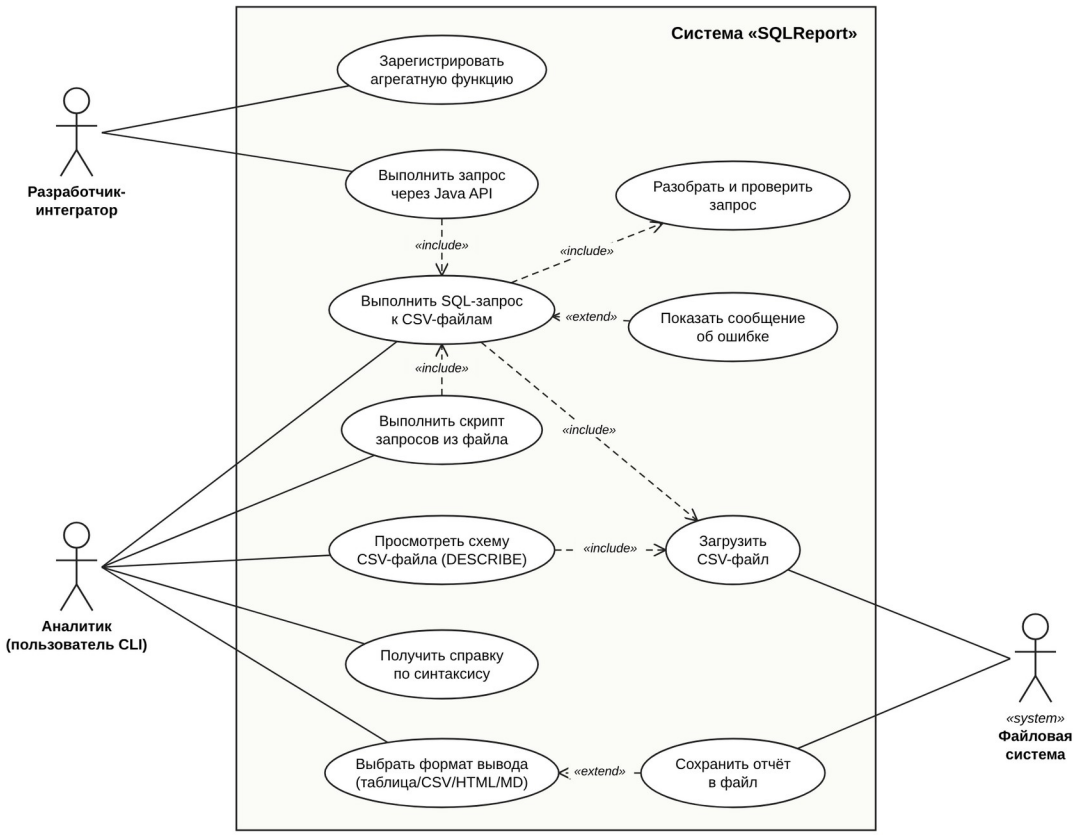


Рисунок 1 — Диаграмма вариантов использования системы SQLReport

Спецификация основного варианта использования приведена в таблице ниже.

Таблица 9 — Спецификация варианта использования UC-1

Элемент	Описание
Название	UC-1. Выполнить SQL-запрос к CSV-файлам
Основной актор	Аналитик (пользователь CLI)
Второстепенный актор	Файловая система (источник CSV-файлов)
Предусловия	Приложение запущено в режиме REPL; файлы, указанные в запросе, существуют и доступны на чтение.
Постусловия (успех)	Пользователь видит отчёт в выбранном формате; исходные файлы не изменены; загруженные таблицы помещены в кеш сеанса.
Основной поток	1) Пользователь вводит запрос и завершает его «;». 2) Система выполняет лексический и синтаксический разбор (включение UC «Разобрать и проверить запрос»). 3) Система загружает упомянутые CSV-файлы или берёт их из кеша (включение UC «Загрузить CSV-файл»). 4) Система проверяет семантику: наличие столбцов, корректность GROUP BY, типы. 5) Система строит план из операторов и выполняет его. 6) Система форматирует результат и выводит его вместе со статистикой «N строк, t мс».
Альтернативные потоки	2а) Синтаксическая ошибка → расширение «Показать сообщение об ошибке»: вывод позиции и подсказки, возврат к шагу 1. 3а) Файл не найден или нарушен формат CSV → сообщение с именем файла и номером строки, возврат к шагу 1. 4а) Неизвестный столбец → сообщение с ближайшим по написанию именем, возврат к шагу 1. 6а) Пользователь указал --out → расширение «Сохранить отчёт в файл».
Связанные требования	US-01…US-06; FR-06…FR-16, FR-21; NFR-1, NFR-4, NFR-6

Соответствие вариантов использования требованиям: «Выполнить SQL-запрос» — US-01…US-05; «Выполнить скрипт» — US-10; «Просмотреть схему» — US-07; «Получить справку» — NFR-7; «Выбрать формат вывода» и «Сохранить отчёт» — US-08; «Выполнить запрос через Java API» — US-11; «Зарегистрировать агрегатную функцию» — US-12; «Показать сообщение об ошибке» — US-06.

4.2. Диаграмма классов

Диаграмма классов представляет статическую структуру системы. Из-за размера она разделена на три части по пакетам: приложение, разбор запроса, AST и вывод (рисунок 2); выполнение запроса (рисунок 3); модель данных (рисунок 4). Классы из других частей показаны в свёрнутом виде (только имя). Для краткости у классов-реализаций (операторов, функций, форматтеров) атрибуты и методы не показаны — они реализуют методы соответствующих интерфейсов.

Обозначения: сплошная линия с ромбом — композиция (закрашенный) или агрегация (пустой); пунктир с пустым треугольником — реализация интерфейса; сплошная линия с пустым треугольником — наследование; пунктир со стрелкой — зависимость (стереотипы «create», «use», «throw»); сплошная линия со стрелкой — направленная ассоциация. Видимость: + public, − private, # protected.

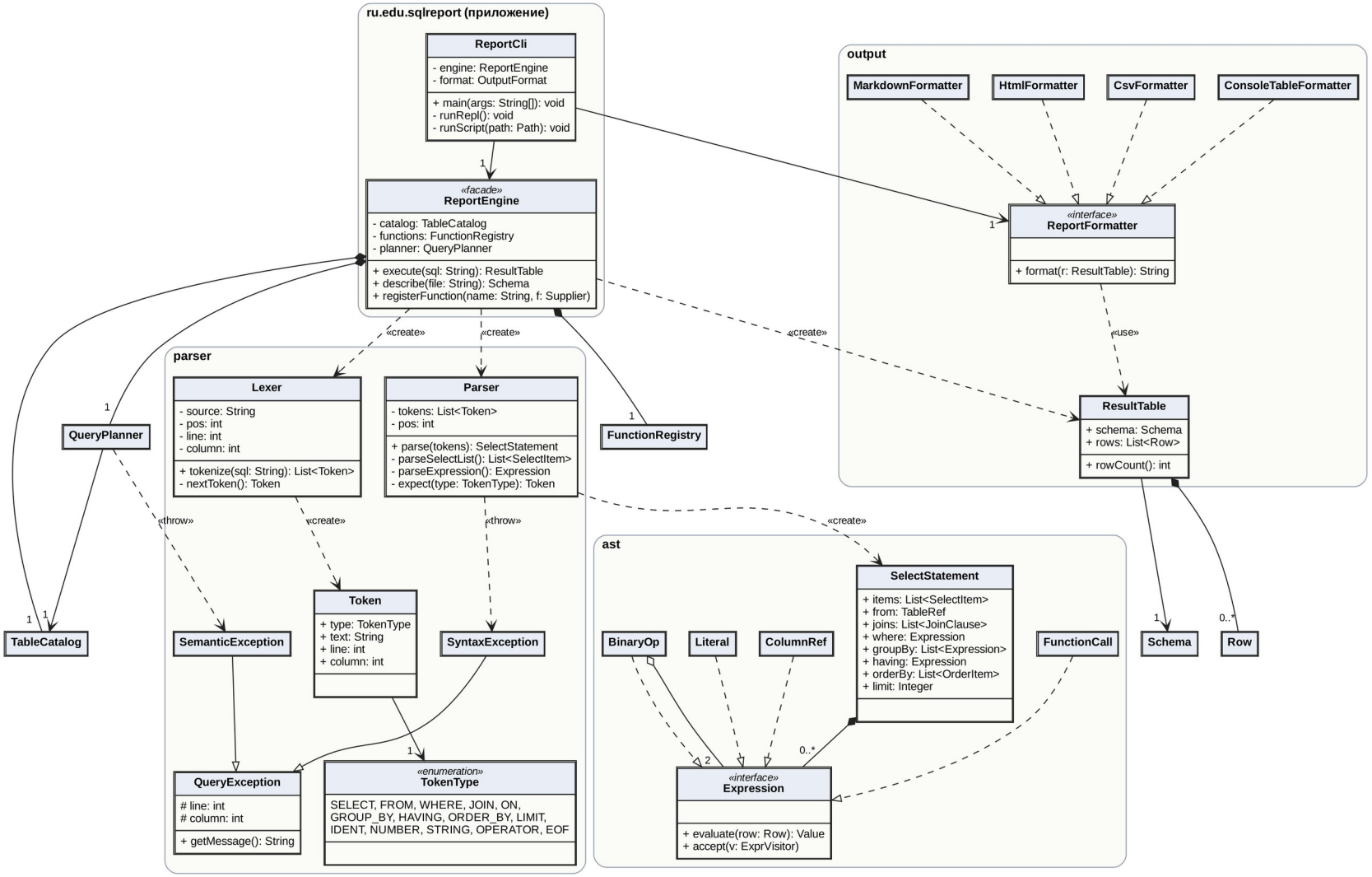


Рисунок 2 — Диаграмма классов, часть 1: приложение, разбор запроса, AST, вывод

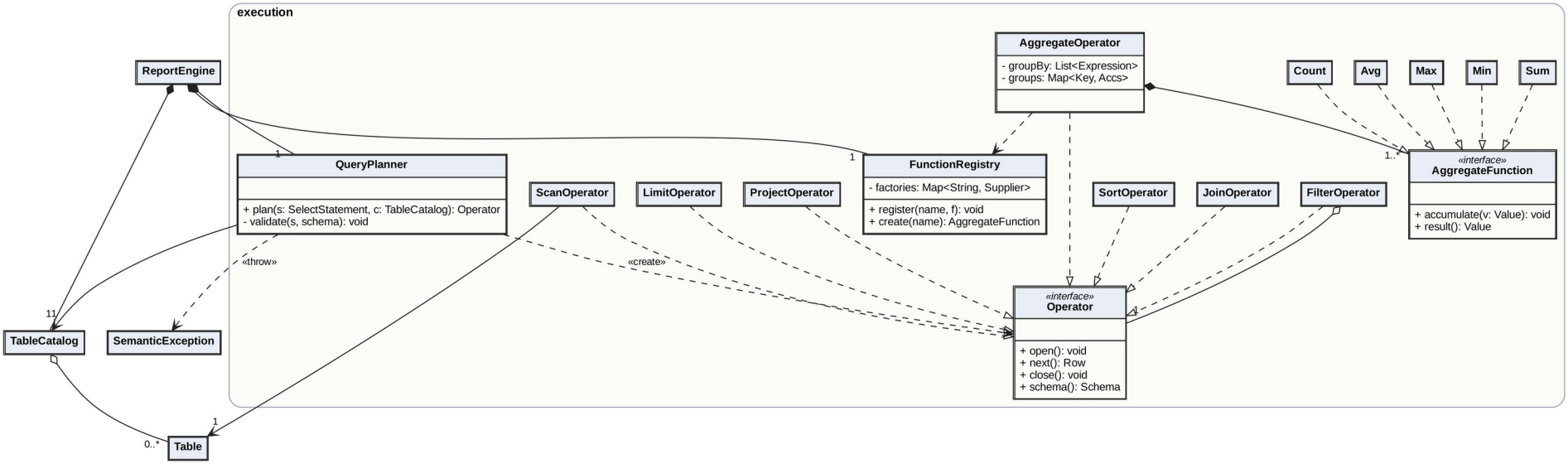


Рисунок 3 — Диаграмма классов, часть 2: выполнение запроса

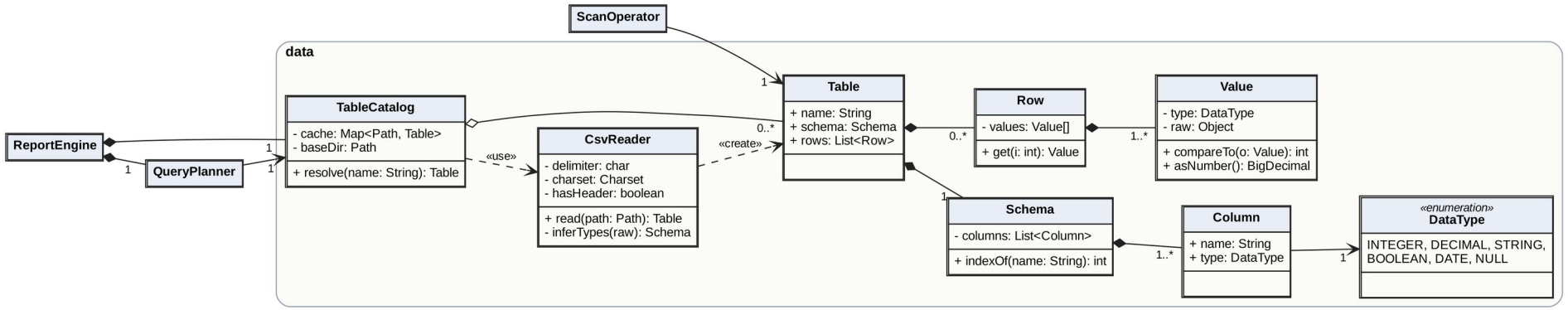


Рисунок 4 — Диаграмма классов, часть 3: модель данных

Ключевые архитектурные решения, отражённые на диаграмме классов:

Таблица 10 — Применённые паттерны проектирования

Паттерн	**Где применён**	**Назначение**
Facade	ReportEngine	Единая точка входа для CLI и внешнего кода: скрывает лексер, парсер, планировщик, каталог таблиц.
Interpreter	Expression и его реализации	Узлы AST сами вычисляют своё значение для строки: evaluate(row).
Iterator (модель Volcano)	Operator: open() / next() / close()	Операторы плана соединяются в конвейер и передают строки по одной; легко добавлять новые операторы.
Composite / Decorator	FilterOperator, SortOperator и др.	Каждый оператор содержит дочерний оператор (агрегация «1»), образуя дерево плана.
Strategy	ReportFormatter, AggregateFunction	Взаимозаменяемые алгоритмы форматирования и агрегирования.
Registry + Factory	FunctionRegistry	Хранит фабрики агрегатных функций по имени; позволяет регистрировать новые функции (FR-20).

Отношения между основными сущностями: ReportEngine композиционно владеет TableCatalog, QueryPlanner и FunctionRegistry (их жизненный цикл совпадает с жизненным циклом движка); Table состоит из Schema и множества Row, Row — из Value; SelectStatement содержит выражения Expression; операторы плана реализуют интерфейс Operator и агрегируют дочерние операторы; SyntaxException и SemanticException наследуются от QueryException.

4.3. Диаграмма последовательности

Диаграмма последовательности (рисунок 5) детализирует основной сценарий UC-1 «Выполнить SQL-запрос к CSV-файлам» для запроса с группировкой. Фрагмент **alt** показывает два варианта: синтаксическая ошибка (исключение передаётся до ReportCli, пользователь видит сообщение с позицией) и корректный запрос. Фрагмент **opt** — чтение файла только при отсутствии таблицы в кеше. Фрагмент **loop** — построчное получение результата из корневого оператора плана (модель Volcano): каждый вызов next() протягивает строку через всю цепочку операторов.

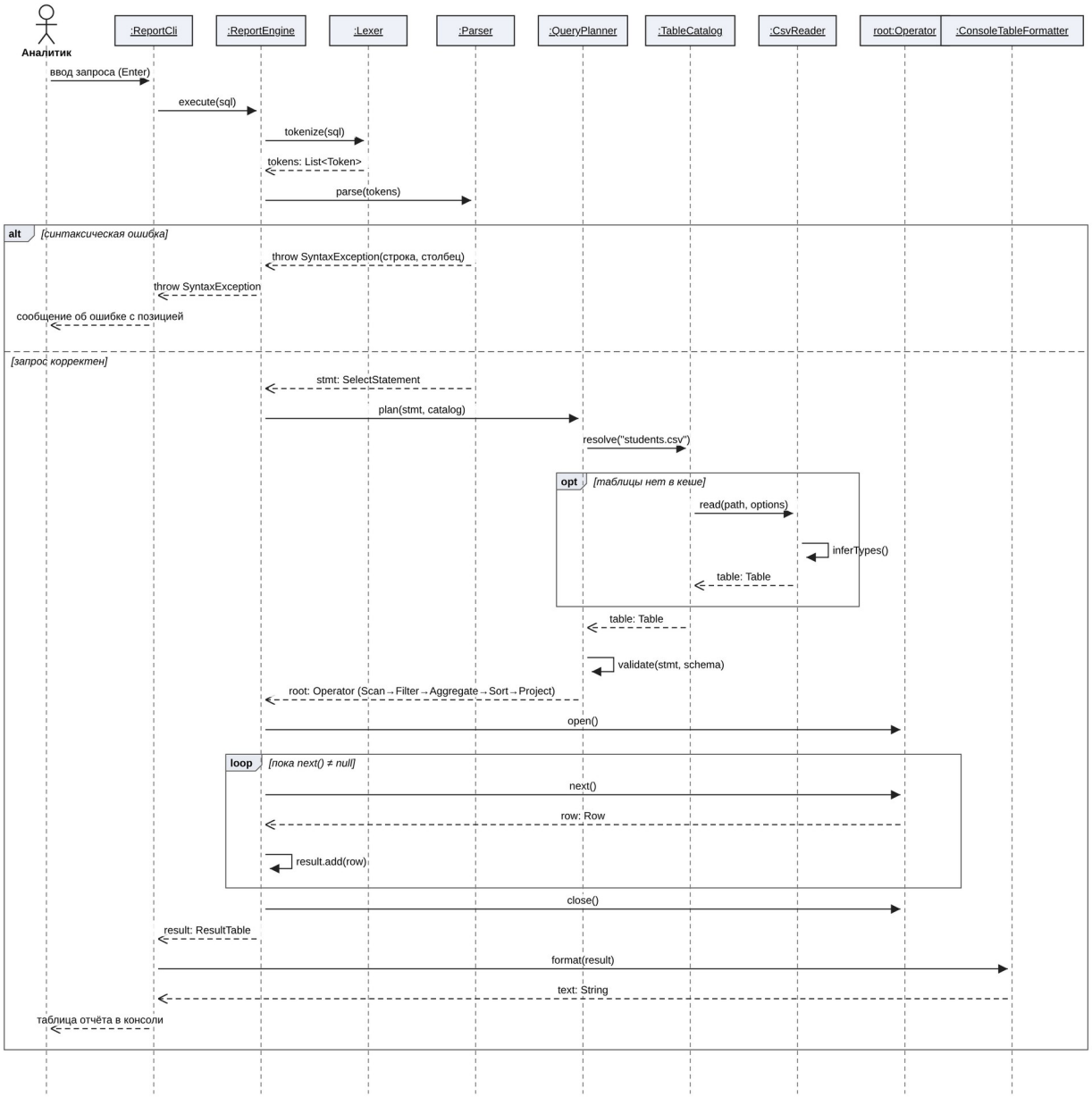


Рисунок 5 — Диаграмма последовательности сценария «Выполнить SQL-запрос»

Сплошные стрелки с закрашенным наконечником — синхронные вызовы, пунктирные — возвраты значений или исключений; петля на линии жизни — вызов собственного метода объекта.